Unit 05: Basics C Language

CONTENTS

Objectives

After studying this unit, you will be able to:

- Understand Programming language
- Discuss C Programming
- Different stages in program development using code blocks IDE
- Programing methodologies
- Character Set, Identifiers and Keywords
- Constants and Variables.

Introduction

Computer is an electronic device which works on the instructions provided by the user. As the computer does not understand natural language, it is required to provide the instructions in some computer understandable language. Such a computer understandable language is known as Programming language.

A computer programming language consists of a set of symbols and characters, words, and grammar rules that permit people to construct instructions in the format that can be interpreted by the computer system Computer Programming is the art of making a computer do what you want it to do. Computer programming is a field that has to do with the analytical creation of source code that can be used to configure computer systems. Computer programmers may choose to function in a broad range of programming functions, or specialize in some aspect of development, support, or maintenance of computers for the home or workplace. Programmers provide the basis for the creation and ongoing function of the systems that many people rely upon for all sorts of information exchange, both business related and for entertainment purposes.

5.1 Programming Language

Different programming languages support different styles of programming. The choice of language used is subject to many considerations, such as company policy, suitability to task, availability of third-party packages, or individual preference. Ideally, the programming language best suited for the task at hand will be selected. Trade-offs from this ideal involve finding enough programmers who know the language to build a team, the availability of compilers for that language, and the efficiency with which programs written in a given language execute.

The basic instructions of programming language are:

1. Input: Get data from the keyboard, a file, or some other device.

2. Output: Display data on the screen or send data to a file or other device.

3. Math: Perform basic mathematical operations like addition and multiplication.

4. Conditional execution: Check for certain conditions and execute the appropriate sequence of statements.

5. Repetition: Perform some action repeatedly, usually with some variation.

5.2 Machine Level Language

Computer language, also known as machine code, is a low-level programming language made up of binary digits (ones and zeros). Before a computer can run code written in high-level languages like Swift and C++, the code must be converted into machine language.

Since computers are digital devices, they only recognize binary data. Every program, video, image, and character of text is represented in binary. This binary data, or machine code, is processed as input by the CPU. The resulting output is sent to the operating system or an application, which displays the data visually. For example, the ASCII value for the letter "A" is 01000001 in machine code, but this data is displayed as "A" on the screen. An image may have thousands or even millions of binary values that determine the color of each pixel.

While computers can be programmed to understand a variety of computer languages, there is only one language that the computer understands without the use of a translation program; this language is known as the computer's machine language or machine code.Machine code is the fundamental language of a computer and is normally written as strings of binary 1s and 0s. The circuitry of a computer is wired in such a way that it immediately recognizes the machine language and converts it into the electrical signals needed to run the computer. An instruction prepared in any language has a two part. The first part is command or operation, and it tells the computer what function to perform. Every computer has an operation code or op-code for each of its functions. The second part of the instruction is the operand, and it tells the computer where to find or store the data or other instructions that are to be maintained. Thus, each instruction tells the control unit of the CPU what to do and the length and location of the data field are involved in the operation. Typical operations involve reading, adding, subtracting, writing and so on.

Task: Discuss Example of Machine Level Language

Instruction Format

We already know that all commuters use binary digits (0s and 1s) for performing operations. Hence, most computers machine language consists of strings of binary numbers and is the only one the CPU directly understands. When stored inside the computer, the symbols which make up the machine language program are made up of 1s and 0s.

 **Example:** A typical program instruction to print out a number on the printer might be.

10110011111101001110110011000011100 1

The program to add two numbers in memory and print the result look something like the Following:

00100000000000110011100 1

00111100000111111000111

10011110001110110011010 1

10110001010101010111000 0

00000000000000000000000 0

This is obviously not a very easy language to learn, partly because it is difficult to read and understand and partly because it is written in a number system with which we are not familiar. But it will be surprising to note that some of the first programmers, who worked with the first few computers, actually wrote their programs in binary form as above. Since human programmers are more familiar with the decimal number system, most of them preferred to write the computer instructions in decimal, and leave the input device to convert these to binary. In fact, without too much effort, a computer can be wired so that instead of using long numbers. With this change, the preceding program appears as follows:

10001471

14002041

30003456

50773456

00000000

The set of instruction codes, whether in binary or decimal, which can be directly understood by the CPU of a computer without the help of a translating program, is called a machine code or machine language. Thus, a machine language program need not necessarily be coded as strings of binary digits (1s and 0s). It can also be written using decimal digits if the circuitry of the computer being used permits this.

Advantages and Limitations of Machine Language

Programs written in machine language can be executed very fast by the computer. This is mainly because machine instructions are directly understood by the CPU writing a program in machine language has several disadvantages which are discussed below.

1. Machine dependent

Because the internal design of every type of commuter is different from every other type of computer and needs different electrical signals to operate, the machine language also is different from computer to computer. It is determined by the actual design or construction of the LU, the control unit, and the size as well as the word length of the memory unit. Hence, suppose after becoming proficient in the machine code of a particular computer, a company decides to change to another computer, the programmer may be required to learn a new machine language and would have to rewrite all the existing programs.

2. Difficult to program

Although easily used by the computer, machine language is difficult to program, it is necessary for the programmer either to memorize the dozens of code numbers for the commands in the machine's instruction set or to constantly refer to keep track of the storage location of data and instructions. Moreover, a machine language programmer must be an expert who knows about the hardware structure of the computer.

3. Error code

For writing programs in machine language, since a programmer has to remember the opcodes and he must also keep track of the storage location of data and instructions, it becomes very difficult for him to concentrate fully on the logic of the problem. This frequently results in program errors. Hence, it is easy to make errors while using machine code.

4. Difficult to modify

It is difficult to correct or modify machine language programs. Checking machine instructions to locate errors is about as tedious as writing them initially. Similarly, modifying a machine language program at a later date is so difficult that many programmers would prefer to code the new logic afresh instead of incorporating the necessary modifications in the old program.

5.3 Assembly Language

Assembly languages are also known as second generation languages. These languages substitute alphabetic symbols for the binary codes of machine language. In assembly language, symbols are used in place of absolute addresses to represent memory locations. Mnemonics are used for operation code, i.e., single letters or short abbreviations that help the programmers to understand what the code represents.

Example: MOV AX, DX.

Caution: Here mnemonic MOV represents 'transfer' operation and AX, DX are used to represent the registers. One of the first steps in improving the program preparation process was to substitute letter symbols mnemonics for the numeric operation codes of machine language. A mnemonic is any kind of mental trick we use to help us remember. Mnemonics come in various shapes and sizes, all of them useful in their own way.

All computers have the power of handling letters as well as numbers. Hence, a computer can be taught to recognize certain combination of letter or numbers. It can be taught to substitute the number 14 every time it sees the symbol ADD, substitute the number 15 every time it sees the symbol SUB, and so forth. In this way, the computer can be trained to translate a program written with symbols instead of numbers into the computer's own machine language. Then we can write program for the computer using symbols instead of numbers, and have the computer do its own translating. This makes it easier for the programmer, because he can use letters, symbols, and mnemonics instead of numbers for writing his programs.

Example: The preceding program that was written in machine language for adding two numbers and printing out the result could be written in the following way:

```
CLA   A
ADD   B
STA   C
TYP   C
```

Which would mean "take A, add B, store the result in C, type C, and halt." The computer by means of a translating program, would translate each line of this program into the corresponding machine language program.

Advantages of Assembly Language

1. Assembly language is easier to use than machine language.

2. An assembler is useful for detecting programming errors.

3. Programmers do not have to know the absolute addresses of data items.

4. Assembly languages encourage modular programming.

Disadvantages of Assembly Language

1. Assembly language programs are not directly executable.

2. Assembly languages are machine dependent and, therefore, not portable from one machine to another.

3. Programming in assembly language requires a higher level of programming skill.

5.4 Assembly Program Execution

An assembly program is written according to a strict set of rules. An editor or word processor is used for keying an assembly program into the computer as a file, and then the assembler is used to translate the program into machine code.

There are two ways of converting an assembly language program into machine language:

1. Manual assembly

2. By using an assembler.

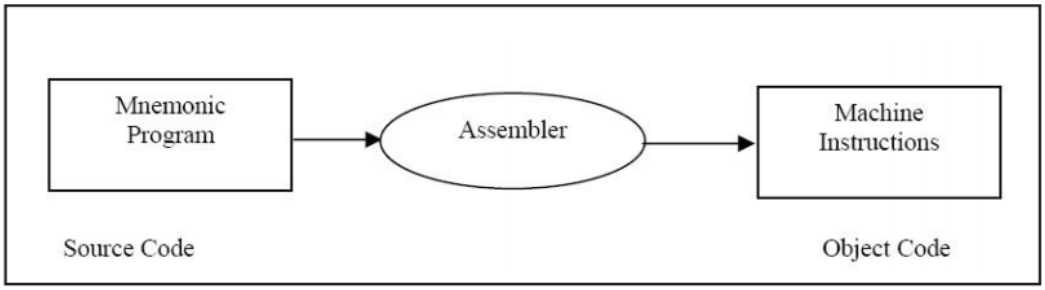
Manual Assembly

It was an old method that required the programmer to translate each opcode into its numerical machine language representation by looking up a table of the microprocessor instructions set, which contains both assembly and machine language instructions. Manual assembly is acceptable for short programs but becomes very inconvenient for large programs. The Intel SDK-85 and most of the earlier university kits were programmed using manual assembly.

Using an Assembler

The symbolic instructions that you code in assembly language is known as - Source program.

An assembler program translates the source program into machine code, which is known as object program.



The steps required to assemble, link and execute a program are:

1. The assembly step involves translating the source code into object code and generating an intermediate .OBJ (object file) or module. The assembler also creates a header immediately in front of the generated .OBJ module; part of the header contains information about incomplete addresses. The .OBJ module is not quite in executable form.

2. The link step involves converting the .OBJ module to an .EXE machine code module. The linker's tasks include completing any address left open by the assembler and combining separately assembled programs into one executable module.

Caution: The linker

(a) Combines assembled module into one executable program

(b) Generates an .EXE module and initializes with special instructions to facilitate its

Subsequent loading for execution.

3. The last step is to load the program for execution. Because the loader knows where the program is going to load in memory, it is now able to resolve any remaining address still left incomplete in the header. The loader drops the header and creates a program segment prefix (PSP) immediately before the program is loaded in memory.

Tools Required for Assembly Language Programming

The tools of the assembly process described may vary in details.

The editor is a program that allows the user to enter, modify, and store a group of instructions or text under a file name. The editor programs can be classified in two groups.

1. Line editors

2. Full screen editors

Line editors, such as EDIT in MS DOS, work with the manage one line at a time. Full screen editors, such as Notepad, WordPad etc. manage the full screen or a paragraph at a time. To write text, the user must call the editor under the control of the operating system. As soon as the editor program is transferred from the disk to the system memory, the program control is transferred from the operating system to the editor program. The editor has its own command and the user can enter and modify text by using those commands. Some editor programs such as WordPerfect are very easy to use. At the completion of writing a program, the exit command of the editor program will save the program on the disk under the file name and will transfer the control to the operating system. If the source file is intended to be a program in the 8086 assembly language the user should follow the syntax of the assembly language and the rules of the assembler.

Linker

For modularity of your programs, it is better to break your program into several sub routines. It is even better to put the common routine, like reading a hexadecimal number, writing hexadecimal number, etc., which could be used by a lot of your other programs into a separate file. These files are assembled separately. After each file has been successfully assembled, they can be linked together to form a large file, which constitutes your complete program. The file containing the common routines can be linked to your other program also. The program that links your program is called the linker.

Loader

Loader is a program which assigns absolute addresses to the program. These addresses are generated by adding the address from where the program is loaded into the memory to all the offsets. Loader comes into action when you want to execute your program. This program is brought from the secondary memory like disk. The file name extension for loading is .exe or .com, which after loading can be executed by the CPU.

Differences between Machine-Level language and Assembly language

Machine-level language	Assembly language
The machine-level language comes at the lowest level in the hierarchy, so it has zero	The assembly language comes above the machine language means that it has less

abstraction level from the hardware.	abstraction level from the hardware.
It cannot be easily understood by humans.	It is easy to read, write, and maintain.
The machine-level language is written in binary digits, i.e., 0 and 1.	The assembly language is written in simple English language, so it is easily understandable by the users.
It does not require any translator as the machine code is directly executed by the computer.	In assembly language, the assembler is used to convert the assembly code into machine code.
It is a first-generation programming language.	It is a second-generation programming language.

5.5 High Level Languages

We have talked about programming languages as COBOL, FORTRAN and BASIC. They are called high level programming languages. The program shown below is written in BASIC to obtain the sum of two numbers.

```
LET     X       =       7
LET     Y       =       10
LET     sum     =       X+Y
PRINT   SUM
END
```

The time and cost of creating machine and assembly languages was quite high. And this was the prime motivation for the development of high level languages. Because of the difficulty of working with low-level languages, high-level languages were developed to make it easier to write computer programs. High level programming languages create computer programs using instructions that are much easier to understand than machine or assembly language code because you can use words that more clearly describe the task being performed.

When writing a program in a high-level language, then the whole attention needs to be paid to the logic of the problem. A compiler is required to translate a high-level language into a low-level language.

Example: High-level languages include FORTRAN, COBOL, BASIC, PASCAL, C, C++ and JAVA.

Advantages of a high-level language

1. Readability: Programs written in these languages are more readable than assembly and machine language.

2. Portability: Programs could be run on different machines with little or no change. We can, therefore, exchange software leading to creation of program libraries.

3. Easy debugging: Errors could easily be removed (debugged).

4. Easy Software development: Software could easily be developed. Commands of programming language are similar to natural languages (ENGLISH).

Differences between Low-Level language and High-Level language

Low-level language	High-level language
It is a machine-friendly language, i.e., the computer understands the machine language, which is represented in 0 or 1.	It is a user-friendly language as this language is written in simple English words, which can be easily understood by humans.
The low-level language takes more time to execute.	It executes at a faster pace.
It requires the assembler to convert the assembly code into machine code.	It requires the compiler to convert the high-level language instructions into machine code.
The machine code cannot run on all machines, so it is not a portable language.	The high-level code can run all the platforms, so it is a portable language.
It is memory efficient.	It is less memory efficient.
Debugging and maintenance are not easier in a low-level language.	Debugging and maintenance are easier in a high-level language.

5.6 Introduction to C Programming

The programming language C was originally developed by Dennis Ritchie of Bell Laboratories and was designed to run on a PDP-11 with a UNIX operating system. Although it was originally intended to run under UNIX, there has been a great interest in running it under the MS-DOS operating system on the IBM PC and compatibles. It is an excellent language for this environment because of the simplicity of expression, the compactness of the code, and the wide range of applicability. Also, due to the simplicity and ease of writing a C compiler, it is usually the first high level language available on any new computer, including microcomputers, mini computers, and mainframes. It allows the programmer a wide range of operations from high level down to a very low level, approaching the level of assembly language. There seems to be no limit to the flexibility available.

Origin and Development of C Language

C is a general-purpose, structured programming language. Structured Languages have a characteristic program structure and associated set of static scope rules. C was originated in Bell Telephone Laboratories presently known as AT & T Bell Laboratories by Dennis Ritchie in 1970. The Kernighan and Ritchie description is commonly referred to as "K&R C". Following the publication of the K & R description, computer professionals, impressed with C's many desirable features, began to promote the use of the language. Since 1980's, the popularity of C has become widespread. The American National Standards Institute (ANSI) proposed a standardized definition of the C language (ANSI committee X3J11). Most commercial C compilers and interpreters are expected to adopt the ANSI standard.

C has the feature of high level programming language as well as the low-level programming. It works as a bridging gap between machine language and the more conventional high-level languages. This feature of C Language made it most popular for system programming as well as application programming.

5.7 Applications of C Language

Mainly C Language is used for Develop Desktop application and system software. Some application of C language are given below.

- C programming language can be used to design the system software like operating system and Compiler.

- To develop application software like database and spread sheets.
- For Develop Graphical related application like computer and mobile games.
- To evaluate any kind of mathematical equation use c language.
- C programming language can be used to design the compilers.
- UNIX Kernel is completely developed in C Language.
- For Creating Compilers of different Languages which can take input from other language and convert it into lower level machine dependent language.
- C programming language can be used to design Operating System.
- C programming language can be used to design Network Devices.
- To design GUI Applications. Adobe Photoshop, one of the most popularly used photo editors since olden times, was created with the help of C.

Evolution of C

By the late fifties, there were many computer languages into existence. However, none of them were general purpose. They served better in a particular type of programming application more than others. Thus, while FORTRAN was more suited for engineering programming, COBOL was better for business programming. At this stage people started thinking that instead of learning so many languages for different programming purposes, why not have a single computer language that can be used for programming any type of application.

In 1960, to this end, an international committee was constituted which came out with a language named ALGOL-60. This language could not become popular because it was too general and highly abstract.

In 1963, a modified ALGOL-60 by reducing its generality and abstractness, a new language, CPL (Combined Programming Language) was developed at Cambridge University. CPL, too turned out to be very big and difficult to learn.

In 1967, Martin Richards, at Cambridge University, stripped down some of the complexities from CPL retaining useful features and created BCPL (Basic CPL). Very soon it was realized that BCPL was too specific and much too less powerful.

In 1970, Ken Thompson, at AT&T labs. Developed a language known by the name B as another simplification to CPL. B, too, like its predecessors, turned out to be very specific and limited in application.

In 1972, Ritchie, at AT&T, took the best of the two BCPL and B, and developed the language C. C was truly a general purpose language, easy to learn and very powerful.

In 1972, Ritchie, at AT&T, took the best of the two BCPL and B, and developed the language C. C was truly a general purpose language, easy to learn and very powerful.

Task: Give two examples of high level languages.

5.8 Compiler and Interpreter

Note that the only language a digital computer understands is binary coded instructions. Even the above implementation will not execute on a computer without further translation into binary (machine) code. This translation is not done manually, however. There are programs available to do this job. These translation programs are called compilers and interpreters.

Compilers and interpreters are programs that take a program written in a language as input and translate it into machine language. Thus a program that translates a C program into machine code is called C compiler; BASIC program into machine code is called a BASIC compiler and so on.

Compilers and interpreters are programs that take a program written in a language as input and translate it into machine language. Thus a program that translates a C program into machine code is called C compiler; BASIC program into machine code is called a BASIC compiler and so on.

A number of different compilers are available these days for C language. GCC, ANSI, Borland C, Turbo C, etc. are only few of the popular C compilers. As a matter of fact, these software tools are little more than just compiler. They provide a complete environment for C program development. They include, among others, an editor to allow Program writing, a Compiler for compilation of the same, a debugger for debugging/testing the program, and so forth. Such tools are referred to as IDE (Integrated Development Environment) or SDK (Software Development Kit).

 **Example:** Code blocks is an IDE for running C and C++ programs on different operating systems like Windows, Linux and Mac OS.

5.9 Program Development in C

The development of a "C" program involves the use of the following programs in the order of their usage.

Editor

This program is used for writing the Source Code, the first thing that any programmer writing a program in any language would be doing.

Debugger

This program helps us identify syntax errors in the source code.

Pre Processor

There are certain special instructions within the source code identified by the # symbol that are carried on by a special program called a preprocessor.

Compiler

The process of converting the C source code to machine code and is done by a program called Compiler.

Linker

The machine code relating to the source code you have written is combined with some other machine code to derive the complete program in an executable file. This is done by a program called the linker.

Writing a C Program

The following rules are applicable to all C-statements:

1. Blank spaces may be inserted between two words to improve the readability of the statement. However, no blank space is allowed within a word.

2. Most of the C-compilers are case-sensitive, and hence statements are entered in small case letters.

3. C has no specific rules about the position at which different parts of a statements be written. Not only can a C statement be written anywhere in a line, it can also be split over multiple lines. That is why it is called free-format language.

4. A C-statement ends with a semi-colon (;)

5. Every C program contains one main() function.

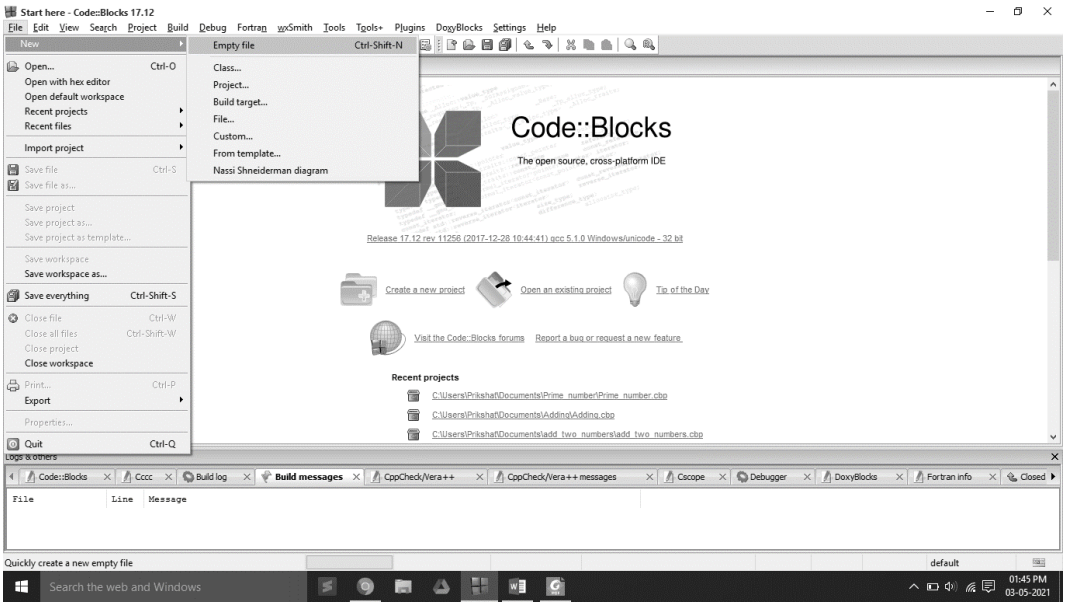
C Program Code

```
#include<stdio.h>
main(){
```

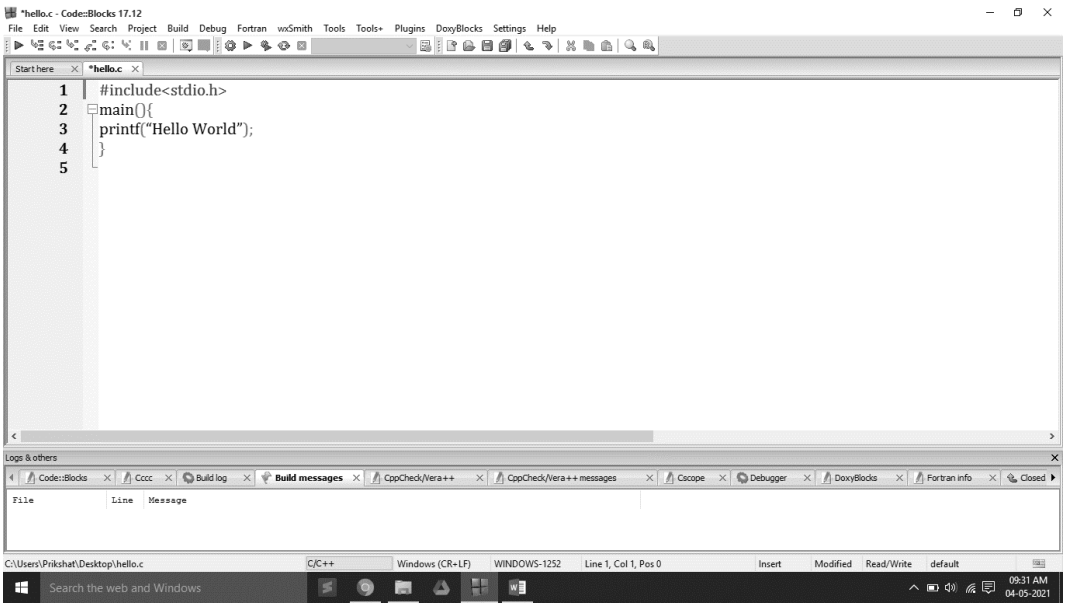
printf("Hello World");
}

Creating and Compiling a C Program

Creating a compiling a C program on operating system use compiler and an integrated development environment. Code blocks is used for create and execute program of C language. File name is hello.c and save in windows operating system using code blocks IDE.



Click on empty file link and save that file with name hello.c and write code.



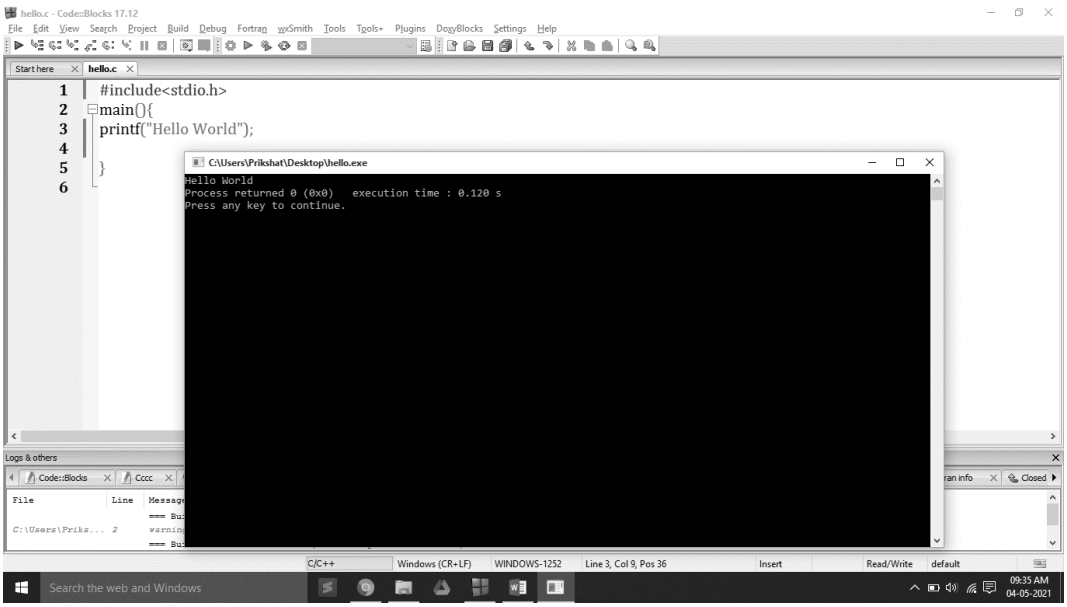
After write code in code blocks,

Gcc compiler is used for compiling code using code blocks

For compile press CTRL+F9 or click on build option and click on build

To run C program in code blocks after write code press first compile program than run (CTRL+F10).

Output will be



5.10 C Character set

Like each other language, 'C' additionally has its own character set. A program is a bunch of directions that, when executed, produce a yield. The information that is prepared by a program comprises of different characters and images. The yield produced is additionally a mix of characters and images.

A character set in 'C' is divided into

- Letters
- Numbers
- Special characters
- White spaces (blank spaces)

Letters

- Uppercase characters (A-Z)
- Lowercase characters (a-z)

Numbers

- All the digits from 0 to 9

White spaces

- Blank space
- New line
- Carriage return
- Horizontal tab

Special characters

- Special characters in 'C' are shown in the given table,

, (comma)	{ (opening curly bracket)
. (period)	} (closing curly bracket)
; (semi-colon)	[(left bracket)
: (colon)	] (right bracket)
? (question mark)	((opening left parenthesis)
' (apostrophe)	) (closing right parenthesis)
" (double quotation mark)	& (ampersand)
! (exclamation mark)	^ (caret)
|(vertical bar)	+ (addition)
/ (forward slash)	- (subtraction)
\ (backward slash)	* (multiplication)
~ (tilde)	/ (division)
_ (underscore)	> (greater than or closing angle bracket)
$ (dollar sign)	< (less than or opening angle bracket)
% (percentage sign)	# (hash sign)
, (comma)	{ (opening curly bracket)

5.11 Identifiers and Keywords

Keywords have fixed meanings, and the meaning cannot be changed. They act as a building block of a 'C' program. There are a total of 32 keywords in 'C'. Keywords are written in lowercase letters.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	short	float	unsigned
continue	for	signed	void

An identifier is nothing but a name assigned to an element in a program. Example, name of a variable, function, etc.

5.12 Constants in C

A constant is a non-changing token with a fixed value. It can be stored in a place in the computer's memory and accessed using that memory address. In C, constants are divided into four categories: integer constants, floating-point constants, character constants, and string constants. Constants can be present in composite forms.

Integer and floating-point constants represent numbers. They are often referred to collectively as numeric-type constants.

C imposes the following rules while creating a numeric constant type data item:

1. Commas and blank spaces cannot be included within the constants.
2. The constant can be preceded by a minus (-) sign if desired.
3. Value of a constant cannot exceed specified maximum and minimum bounds. For each type of constant, these bounds will vary from one C-compiler to another.

Constants are the fixed values that remain unchanged during the execution of a program and are used in assignment statements. Constants can also be stored in variables.

The declaration for a constant in C language takes the following form:

<Data type ><variable_name> = <value>

Example: float pi = 3.14

This declaration defines a constant named pi whose value remains 22/7 throughout the program in which it is defined.

C language facilitates five different types of constants.

1. Character
2. Integer
3. Real
4. String
5. Logical

Character Constants

A character constant consists of a single character, single digit, or a single special symbol enclosed with in a pair of single inverted commas. The maximum length of a character constant is one character.

Example: 'a' is a character constant

Example: 'd' is a character constant

Example: 'P' is a character constant

Example: '7' is a character constant

Example: '*' is a character constant

Integer Constants

An integer constant refers to a sequence of digits and has a numeric value. There are three types of integers in C language: decimal, octal and hexadecimal.

Decimal integers 1, 56, 7657, -34 etc.

Octal integers 076, -076, 05 etc. (preceded by zero, 0)

Hexadecimal integers 0x56, -0x5D etc. (preceded by zero, 0x)

No commas or blanks are allowed in integer constants.

String Constants

A string constant is a sequence of one or more characters enclosed within a pair of double quotes (""). If a single character is enclosed within a pair of double quotes, it will also be interpreted as a string constant and not a character constant.

Example: 1. " Hello World"

2. "A"

Actually, a string is an array of characters terminated by a NULL character. Thus, "a" is a string consisting of two characters, viz. 'a' and NULL('\0').

Logical Constants

The value of a logical constant may be true or false. In C, a non-zero value is considered true, while 0 is considered false.

5.13 Variables

A variable is an object whose value can change while a programme is running. A variable is a symbolic representation of the address in memory space where values can be stored, accessed, and updated. Each variable has a memory location or address assigned to it, and the value of that variable is stored there.

Each variable has its own name, data type, size, and value. All variables must have a type so that the compiler can record all necessary information about them, generate the appropriate code during translation, and allocate the necessary memory space.

Every programming language has its own set of rules that must be observed while writing the names of variables. If the rules are not followed, the compiler reports compilation error. Rules for Constructing Variable Name in C language are listed below:

1. Variable name may be a combination of alphabets, digits or underscores. Sometimes, an additional constraint on the number of characters in the name is imposed by compilers in which case its length should not exceed 8 characters.

2. First character must be an alphabet or an underscore (_).

3. No commas or blank spaces are allowed in a variable name.

4. Among the special symbols, only underscore can be used in a variable name. E.g.: emp_age, item_4, etc.

5. No word, having a reserved meaning in C can be used for variable name

Declaration of Variables

C language is strongly typed language, meaning that all the variables must be declared before their use. Declaration does two things:

1. It tells the compiler what the variable name is.

2. It specifies what type of data the variable will hold

In C language, a variable declaration has the form:

<Type-specifier><comma-separated-list-of-variables>;

Here <type-specifier> is one of the valid data types (e.g. int, float, char, etc.). List-of-variables is a comma-separated list of identifiers representing the program variables.

Example: inti, j, k; //creates integer variables i,j and K

Once variable has been declared in the above manner, the compiler creates a space in the memory and attaches the given name to this space. This variable can now be used in the program.

A value is stored in a variable using assignment operation. Assignment is of the form:

<Variable-name> = <value>;

Obviously, before assignment, the variable must be declared.

C also allows assignment of a value to a variable at the time of declaration. It takes the following form:

<Type-specifier><variable_name> = <value>;

e.g. :int I = 5;

Let us consider some of programming examples to illustrate the matter further.

```
        /* Example of assignments */
/* declaration */
int a1, b1;
/* declaration & assignment */
intvar = 5;
int a, b = 6, c;
/* declaration & multiple assignment */
int p, q, r, s;
p = q = r = s = 5;
}
```

values stored in various variables are:

var = 5

a = 0, b = 6

c = garbage value

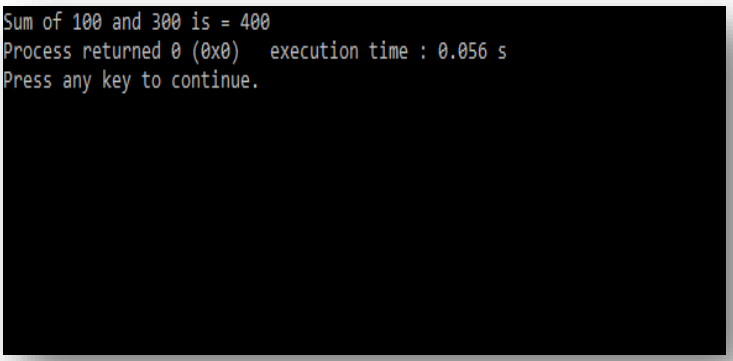
p = 5, q = 5, r = 5, s = 5

Notes: Write a program in C to show the variable declaration.

Example: Write a program to find sum of two numbers.

Solution : - #include<stdio.h>

```
int main(){
int num1,num2,sum;
num1=100;
num2=300;
sum=num1+num2;
printf("Sum of %d and %d is = %d",num1,num2,sum);
return 0;
}
```



Output:-

Summary

- A computer programming language consists of a set of symbols and characters, words, and grammar rules that permit people to construct instructions in the format that can be interpreted by the computer system Computer Programming is the art of making a computer do what you want it to do.
- Machine language, or machine code, is a low-level language comprised of binary digits (ones and zeros).
- Assembly languages are also known as second generation languages. These languages substitute alphabetic symbols for the binary codes of machine language.
- High-level languages include FORTRAN, COBOL, BASIC, PASCAL, C, C++ and JAVA.
- C is a general-purpose, structured programming language. Structured Languages have a characteristic program structure and associated set of static scope rules. C was originated in Bell Telephone Laboratories presently known as AT & T Bell Laboratories by Dennis Ritchie in 1970.

- To develop application software like database and spread sheets.
- For Develop Graphical related application like computer and mobile games.
- Keywords have fixed meanings, and the meaning cannot be changed. They act as a building block of a 'C' program. There are a total of 32 keywords in 'C'
- Code blocks is an IDE for running C and C++ programs on different operating systems like Windows, Linux and Mac OS.
- An identifier is nothing but a name assigned to an element in a program. Example, name of a variable, function, etc.
- A constant is a value that doesn't change throughout the execution of a program.
- A variable is an identifier which is used to store a value.
- A variable is an entity whose value can change during program execution. A variable can be thought of as a symbolic representation of address of the memory space where values can be stored, accessed and changed.

Keywords

Programming: Computer programming is the process of designing and building an executable computer program to accomplish a specific computing result or to perform a specific task.

Machine Level Language: Machine language, or machine code, is a low-level language comprised of binary digits (ones and zeros).

Assembly Level Language: Assembly language is a low-level programming language.

High Level Language: high-level programming language is a programming language with strong abstraction from the details of the computer.

Constant: A named data item whose value does not change throughout the execution of the Program

Variable: A named location in the memory that can store a value of specified type.

Self Assessment

1. A computer program is?

A. a sequence of English language statements.

B. a sequence of images to create a picture.

C. a sequence of instructions to perform an specific task

D. none of above

2. A program that can converts high-level language programs into machine understandable form is known as…..

A. Compiler

B. Sensor

C. Translation

D. None of these

3. An error is also known as

A. Bug

B. Insect

C. Worm

D. virus

4. A high-level language is a
A. Human understandable language with proper syntax to write programs
B. Machine understandable language that can be processed on a machine
C. Both (a) and (b) statements defines it
D. None of these

5. A low-level language is a
A. Human understandable language with proper syntax to write programs
B. Machine understandable language that can be processed on a machine
C. Both (a) and (b) statements defines it
D. None of these

6. Which of the following is type of variable in C language?
A. Local Variable
B. Global Variable
C. All of the above
D. None of the above

7. Which is the only function all C programs must contain?
A. start()
B. system()
C. main()
D. printf()

8. Which of the following is true for variable names in C?
A. Variable can be of any length
B. They can contain alphanumeric characters as well as special characters
C. Reserved Word can be used as variable name
D. Variable names cannot start with a digit

9. What are the entities whose values can be changed called?
A. Constants
B. Variables
C. Modules
D. Tokens

10. Which of the following is not a constant type in C language?
A. Real
B. Integer
C. Main
D. Character

11. Which is not a valid C variable name?
A. int number;

B. float rate;
C. intvariable_count;
D. int $reg_no;

12. The format identifier '%d' is used for _____ data type?
A. Char
B. Int
C. Double
D. Float

Answer for Self Assessment

1.	C	2.	A	3.	A	4.	A	5.	B
6.	C	7.	C	8.	D	9.	B	10	C
11.	D	12.	B						

Review Questions

1. Differentiate between local and global variables with examples.
2. Explain the types of constants in C with examples.
3. How are symbolic constants defined in C? Illustrate with code.
4. Define identifiers and keywords in C with examples.
5. Why are keywords reserved in C? Give at least five examples.
6. What is the C character set? Explain with suitable examples.
7. Why is understanding the character set important in programming?
8. Write and explain the stages of program development in C.
9. How is debugging an important step in program development?
10. Differentiate between compiler and interpreter with examples.
11. Explain the working process of a compiler.
12. What do you understand by a programming language? Explain its types with examples.
13. Discuss the role of programming languages in software development.
14. What is machine-level language? State its advantages and disadvantages.
15. Why is machine language considered difficult for programmers? Explain with an example.
16. Differentiate between machine language and assembly language.
17. Write a short note on mnemonics in assembly language.
18. Explain the steps involved in executing an assembly program.
19. What is the role of an assembler in program execution?
20. Compare high-level languages with low-level languages. Give suitable examples.
21. Why are high-level languages preferred for application development?
22. Discuss the features of C programming language.
23. Why is C called a middle-level language?
24. Explain the real-life applications of C language in different fields.
25. Discuss why C language is still widely used despite the availability of modern languages.

Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C

Web Links

https://www.tutorialspoint.com/index.htm

www.webopedia.com

www.web-source.net